

# Capitolo 12 — Source Precedence — non tutte le informazioni valgono allo stesso modo

|                    |                                                                        |
|--------------------|------------------------------------------------------------------------|
| <b>Pubblico</b>    | Lettori tecnici e non tecnici del WCM Process & Memory Book            |
| <b>Versione</b>    | FROZEN-12                                                              |
| <b>Data master</b> | 2026-08-29                                                             |
| <b>Stato</b>       | FROZEN                                                                 |
| <b>Source path</b> | wcm/documentation/process-memory-book/chapters/12_source_precedence.md |
| <b>Source SHA</b>  | ddff24c                                                                |
| <b>Release</b>     | 2026-08-29T08:41:46.818Z                                               |

*GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.*

---

**Stato:** FROZEN **Parte:** IV — INDEX-FIRST: come WCM trova quello che gli serve **Scope:** WCM generale / domain-agnostic **Review:** Technical Review PASS / Human Comprehension Review PASS

---

## 12.0 Trovare una fonte non basta

Nel Capitolo 11 abbiamo seguito INDEX-FIRST passo per passo.

Abbiamo visto come WCM parte da un punto di ingresso, usa una mappa, apre solo le fonti necessarie e si ferma quando il contesto è sufficiente.

Ma resta un problema decisivo.

Che cosa succede quando troviamo **più fonti che parlano della stessa cosa?**

Una potrebbe essere recente. Una potrebbe essere molto dettagliata. Una potrebbe essere facile da leggere. Un'altra potrebbe essere quella che ha davvero authority sul tema.

Se trattassimo tutte queste fonti come equivalenti, INDEX-FIRST ci aiuterebbe a trovare informazioni, ma non a capire **quali informazioni devono prevalere.**

È qui che entra in gioco la Source Precedence.

**Source Precedence è la disciplina con cui WCM attribuisce peso diverso alle fonti in funzione della domanda, dell'authority, dello status e del tipo di fatto che deve essere verificato.**

Non è una classifica estetica dei documenti.

Non dice quale file è scritto meglio.

Non dice quale file è più nuovo.

Dice quale fonte deve essere considerata prima quando più fonti candidate potrebbero rispondere alla stessa domanda.

---

## 12.1 Il problema delle versioni concorrenti

Immaginiamo un'organizzazione con tre documenti.

Il primo dice:

| *"La procedura prevede A."*

Il secondo dice:

| *"Stiamo valutando B."*

Il terzo, scritto ieri, dice:

| *"Forse sarebbe meglio C."*

Se cercassimo soltanto per parole chiave, tutti e tre potrebbero sembrare pertinenti.

Se scegliessimo il documento più recente, potremmo finire su C.

Se scegliessimo quello più lungo, potremmo finire su B.

Ma se A è la baseline ancora attiva, la risposta operativa resta A.

Questo semplice esempio mostra la differenza tra:

PERTINENZA

≠

AUTHORITY

Una fonte può essere molto pertinente semanticamente e, nello stesso tempo, non avere il diritto di cambiare ciò che il sistema considera corrente.

## 12.2 Più recente non significa più autorevole

La data è importante.

Ma non basta.

Nel WCM una fonte più recente non prevale automaticamente su una fonte più autorevole appartenente a uno strato superiore.

Per esempio, un appunto recente può descrivere un'idea nuova senza averla ancora trasformata in una decisione.

Un report può raccontare un problema appena emerso senza modificare il protocollo che governa il comportamento corrente.

Una projection human-facing può essere aggiornata ieri ma essere comunque stale rispetto a una decisione canonica modificata oggi.

La domanda corretta non è quindi:

| *"Qual è il documento più recente?"*

ma:

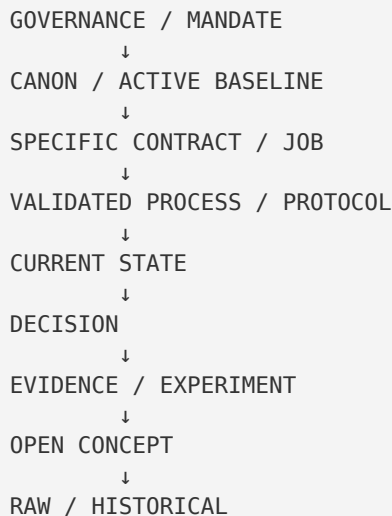
| **"Quale fonte ha authority su questo tipo di informazione, e qual è la sua versione corrente?"**

La recency aiuta a distinguere current da stale.

L'authority aiuta a distinguere ciò che può governare il comportamento da ciò che può soltanto descriverlo, proporlo o documentarlo.

## 12.3 La gerarchia generale

PROT-005 definisce una precedence generale, salvo regole più specifiche del task:



Questa mappa va letta con attenzione.

Non significa che una fonte in alto contenga sempre la risposta.

Significa che, **quando due fonti pretendono di governare la stessa informazione**, lo strato superiore deve essere verificato prima.

E soprattutto significa che la precedence è **task-scoped**.

Se stiamo cercando lo stato esecutivo corrente di un workflow, il runtime strutturato può diventare la fonte giusta per quel fatto specifico.

Se stiamo cercando chi può autorizzare una modifica, il runtime non può inventare l'authority: bisogna risalire a governance, canone o contratto applicabile.

## 12.4 Governance e Mandate

Il livello più alto della precedence generale riguarda le regole che definiscono **chi può fare cosa e come il sistema può evolvere**.

La governance non racconta semplicemente il lavoro.

Definisce il perimetro entro cui il lavoro può essere svolto.

Domande tipiche:

- questa operazione è una RUN o un CHANGE?
- chi possiede l'authority?
- esiste una stop condition obbligatoria?
- una scrittura è consentita autonomamente?
- un gate umano è richiesto?

Una risposta proveniente da un report operativo non può sovrascrivere queste regole.

Un log non può attribuire authority.

Una UI non può diventare governance perché mostra un pulsante.

VISIBILITÀ

≠

AUTHORITY

Nel WCM corrente, il Canon Register indica quali documenti sono autorevoli per le diverse aree del metodo. Questo evita di dover dedurre l'authority dal nome del file o dalla memoria della sessione.

## 12.5 Canon / Active Baseline

Subito sotto governance e mandate troviamo il canone e la baseline attiva.

"Canon" qui non significa verità eterna.

Significa:

**| riferimento attualmente approvato per quella parte del metodo.**

Una baseline può essere sperimentale, in field validation o evolutiva e restare comunque la baseline corrente.

Il suo status dice quanto è matura.

La sua authority dice se deve governare il comportamento presente.

Questa distinzione è importante:

APPROVATO COME BASELINE

≠

DIMOSTRATO UNIVERSALMENTE

WCM può quindi usare una regola come corrente senza sostenere che quella regola sia già stata validata in ogni possibile contesto.

## 12.6 Specific Contract / Authority

Esistono casi in cui una regola generale viene applicata dentro un incarico, un job o un contratto operativo specifico.

Quel contratto può restringere il campo.

Per esempio, una governance generale può dire che un attore è autorizzato a svolgere determinate attività.

Un job specifico può dire:

**|** *"In questa esecuzione opera soltanto su questo perimetro."*

La regola generale resta valida, ma il contratto specifico definisce il confine del lavoro corrente.

Questo non autorizza un contratto inferiore a contraddire governance o canone.

La relazione corretta è:

AUTHORITY SUPERIORE

→ definisce ciò che è consentito

CONTRATTO SPECIFICO  
→ restringe e concretizza il lavoro consentito

Non:

CONTRATTO SPECIFICO  
→ inventa nuova authority

## 12.7 Processi e protocolli validati

Processi e protocolli descrivono come il WCM deve comportarsi in condizioni definite.

Un processo organizza una sequenza operativa.

Un protocollo impone regole, guard, trigger o vincoli.

Quando una domanda è procedurale, questi nodi hanno un peso elevato.

Esempio astratto:

**|** *"Prima di aprire un'altra fonte devo verificare qualcosa?"*

La risposta non va cercata in un vecchio report che racconta come qualcuno si è comportato una volta.

Va cercata nel protocollo corrente che governa il retrieval.

L'evidence può spiegare **perché** il protocollo è nato.

Il protocollo corrente dice **come dobbiamo comportarci adesso**.

## 12.8 Runtime per gli execution facts

Qui serve una precisazione fondamentale.

La precedence generale di PROT-005 colloca il current state sotto processi e protocolli. Ma per i **fatti esecutivi strutturati** WCM applica una gerarchia più specifica.

DEC-013 stabilisce:

```
AUTHORITY / CANON
  ↓
RUNTIME WORKFLOW CHECKPOINTS
  ↓
DERIVED PROJECT STATE
  ↓
PROJECTION
  ↓
READ MODEL
  ↓
UI
```

La parola chiave è:

**| execution facts.**

Se dobbiamo sapere:

- quale transizione è stata completata;
- qual è la **next\_transition**;
- se un workflow è ACTIVE o INTERRUPTED\_RESUMABLE;
- se è stata raggiunta una true stop;
- se serve Resume Priority;

il runtime strutturato è il master dello stato esecutivo.

Una sintesi umana può essere utile.

Ma se dice qualcosa di diverso, non può prevalere sul runtime per quel fatto.

```
RUNTIME DICE X
STATE HUMAN VIEW DICE Y
X ≠ Y
```

```
→ X prevale per l'esecuzione
→ la vista va riconciliata
```

Questo non rende il runtime superiore alla governance.

Il runtime può dire **dove siamo**.

Non può decidere da solo **chi ha authority** o cambiare il significato delle regole che lo governano.

## 12.9 Stato corrente e projection

Una delle confusioni più comuni nasce quando esistono più rappresentazioni dello stesso stato.

Possiamo avere:

- runtime strutturato;
- derived state;
- documento human-facing;
- projection per un pannello;
- read model in un database;
- interfaccia utente.

Sono tutte informazioni utili.

Ma non hanno la stessa funzione.

```
SOURCE OF TRUTH
→ DERIVAZIONE
→ PROJECTION
→ PRESENTAZIONE
```

Una UI può essere perfettamente aggiornata e restare una presentation surface.

Un database usato dal pannello può essere tecnicamente affidabile e restare un read model.

Se una projection contraddice la source of truth, la correzione non consiste nel reinterpretare la source of truth sulla base della UI.

Consiste nel riallineare la projection.

---

## 12.10 Decisioni

Le decisioni hanno una funzione diversa dai processi e dai runtime facts.

Una decisione risponde tipicamente a domande come:

- che cosa è stato scelto?
- da chi?
- con quale authority?
- quale alternativa è stata scartata?
- quale baseline deriva da quella scelta?

Una decisione può quindi essere una fonte estremamente autorevole per il **perché** e per la lineage causale.

Ma anche qui lo status conta.

Una decisione superseded non deve essere trattata come corrente solo perché è formalmente ben scritta.

La storia resta preziosa.

Non governa automaticamente il presente.

---

## 12.11 Evidence

L'evidence risponde a un'altra domanda:

**"Quali fatti, test, osservazioni o risultati supportano questo claim?"**

Può essere fortissima dal punto di vista probatorio.

Ma evidence e authority non sono la stessa cosa.

Un esperimento può dimostrare che una regola corrente ha un limite.

Non per questo modifica automaticamente la regola.

EVIDENCE  
→ può giustificare una revisione

EVIDENCE  
≠  
PROMOZIONE AUTOMATICA A BASELINE

Nel WCM il passaggio da evidenza a baseline richiede il percorso di promotion appropriato e, se il significato materiale cambia, l'authority prevista dal Change Gate.

Questo impedisce a un risultato interessante di trasformarsi silenziosamente in nuova governance.

---

## 12.12 Concept, raw e storico

I concept sono utili per esplorare possibilità.

Il raw è utile per ricostruire fatti grezzi.

Lo storico è utile per capire come siamo arrivati fin qui.

Nessuno di questi strati è inutile.

Il punto è diverso:

**non devono essere scambiati per baseline corrente solo perché contengono informazioni pertinenti.**

Un concept può anticipare una direzione futura.

Un documento storico può contenere una formulazione ancora molto convincente.

Un raw log può mostrare il comportamento reale di un sistema.

Ma se la domanda è:

*"Qual è la regola corrente?"*

queste fonti entrano in gioco soltanto se servono a verificare un conflitto, ricostruire lineage o comprendere il perché.

È il principio L3 on demand visto nel Capitolo 11.

## 12.13 Status matters

Due documenti dello stesso tipo possono avere peso diverso a causa dello status.

Per esempio:

```
PROTOCOLLO A – ACTIVE  
PROTOCOLLO B – SUPERSEDED
```

Entrambi sono protocolli.

Ma non sono equivalenti.

Lo stesso vale per:

- CURRENT vs STALE;
- APPROVED vs PROPOSED;
- FROZEN vs DRAFT;
- ACTIVE vs RETIRED;
- VALIDATED vs OPEN;
- CANONICAL vs WORKING.

Per questo Source Precedence non può basarsi soltanto sul tipo di documento.

Deve combinare almeno:

```
TYPE  
+  
STATUS  
+  
SCOPE
```



## 12.14 Il conflitto tra fonti

Arriviamo al caso più delicato.

Che cosa succede se due fonti autorevoli sembrano contraddirsi?

La tentazione di un sistema cognitivo è mediare.

Prendere un po' dall'una e un po' dall'altra.

Produrre una sintesi plausibile.

Nel WCM questo comportamento è pericoloso.

PROT-005 impone:

### **| No silent conflict resolution.**

Se due fonti autorevoli confliggono materialmente, il sistema non deve inventare una terza regola.

Deve prima capire se il conflitto è solo apparente.

Domande utili:

1. hanno lo stesso scope?
2. parlano dello stesso tipo di fatto?
3. una è superseded?
4. una è una projection e l'altra la source of truth?
5. esiste una regola di precedence specifica per quel dominio?
6. una delle due è stale?

Se il conflitto resta reale:

CONFLITTO AUTOREVOLE NON RISOLTO  
↓  
FAIL CLOSED / ESCALATION

"Fail closed" non significa che il sistema deve bloccarsi per qualsiasi differenza di formulazione.

Significa che, quando una contraddizione materiale impedisce di sapere quale comportamento sia autorizzato, WCM non deve colmare il vuoto con un'interpretazione creativa.

## 12.15 Source Precedence non è una scala assoluta

La gerarchia generale è una mappa di default.

Non è una classifica rigida valida allo stesso modo per ogni domanda.  
Consideriamo tre domande.

## Domanda A

| *"Chi può autorizzare questa modifica?"*

La route sale verso governance, mandate e authority.

## Domanda B

| *"Qual è la prossima transizione del workflow?"*

La route verifica authority/canon e poi usa il runtime strutturato come master dell'execution fact.

## Domanda C

| *"Perché questa regola è stata introdotta?"*

La route può scendere verso decisioni, evidence e storico.

Lo stesso documento può quindi essere fondamentale per una domanda e secondario per un'altra.

La Source Precedence corretta è sempre:

| **autorevolezza rispetto all'informazione che stiamo cercando.**

---

# 12.16 Un esempio completo

Immaginiamo di dover rispondere alla domanda:

| *"Il sistema può continuare autonomamente?"*

Potremmo trovare:

- una nota recente che dice "probabilmente sì";
- una vista human-facing che dice "in corso";
- un runtime che dice **WAITING\_AUTHORITY**;
- una governance che stabilisce che **WAITING\_AUTHORITY** è una vera stop condition.

La Source Precedence porta a questa lettura:

```
GOVERNANCE
→ definisce il significato del gate

RUNTIME
→ dice che il gate è attualmente raggiunto

HUMAN VIEW
→ descrive lo stato, ma non può sovrascrivere il runtime

NOTA RECENTE
→ può essere contesto, non authority
```

Conclusione:

| *il sistema non continua autonomamente.*

Non perché il runtime "vince sempre".

Ma perché governance e runtime rispondono insieme a due domande diverse:

- **che cosa significa questo stato?**
- **qual è lo stato attuale?**

---

## 12.17 Un secondo esempio: evidence contro baseline

Domanda:

| *"Un test recente mostra che la procedura corrente è inefficiente. Possiamo cambiarla subito?"*

Fonti:

- evidence recente e convincente;
- protocollo ACTIVE;
- governance che distingue RUN da CHANGE.

La risposta corretta non è ignorare l'evidence.

Ma non è nemmeno promuoverla automaticamente.

EVIDENCE

→ segnala possibile necessità di modifica

PROTOCOLLO ACTIVE

→ resta comportamento corrente

GOVERNANCE

→ stabilisce il gate necessario per cambiare la baseline

La Source Precedence permette quindi di prendere sul serio l'evidence **senza confondere osservazione e authority**.

---

## 12.18 Un terzo esempio: manuale contro source of truth

Domanda:

| *"Il manuale dice una cosa diversa dalla baseline tecnica. Quale correggiamo?"*

Nel Documentation System WCM i manuali sono living projections.

Non sono authority autonome.

Se una projection contraddice la baseline autorevole:

BASELINE AUTOREVOLE

→ resta source of truth

MANUALE

- STALE
- va riallineato

Il manuale può far emergere un problema reale.

Ma non risolve il conflitto cambiando silenziosamente il metodo.

---

## 12.19 Source Precedence e memoria

La Working Memory può ricordare una decisione correttamente.

Può anche ricordarla in modo incompleto.

Per questo resta valida la regola:

**Memory is not authority.**

La memoria aiuta a formulare la ricerca.

Può suggerire:

**"Credo che la regola corrente sia questa."**

INDEX-FIRST + Source Precedence trasformano quel ricordo in una verifica:

MEMORIA  
→ ipotesi di route  
→ fonte autorevole corrente  
→ verifica

Questo è uno dei passaggi che consente al WCM di usare il valore del contesto vivo senza trasformare il ricordo del sistema in canone implicito.

---

## 12.20 Source Precedence e determinismo

Source Precedence aumenta la ripetibilità del percorso.

Se due agenti devono rispondere alla stessa domanda e applicano la stessa mappa di authority, hanno maggiore probabilità di consultare le stesse fonti rilevanti.

Ma non dobbiamo esagerare il claim.

PRECEDENCE DETERMINA  
→ quali fonti controllare prima  
→ quali fonti non possono prevalere silenziosamente  
  
PRECEDENCE NON DETERMINA DA SOLA  
→ ogni interpretazione semantica  
→ ogni decisione cognitiva complessa

Due agenti possono ancora interpretare diversamente un testo ambiguo.

La differenza è che il disaccordo avviene **sulle stesse fonti autorevoli**, non su due fotografie casuali e incompatibili del sistema.

---

## 12.21 Source Precedence come guard contro il "documento convincente"

Gli esseri umani e i modelli linguistici condividono una vulnerabilità.

Un testo ben scritto può sembrare vero.

Un testo dettagliato può sembrare autorevole.

Un testo recente può sembrare aggiornato.

Un testo tecnico può sembrare normativo.

La Source Precedence separa la forza retorica dalla forza organizzativa.

CONVINCENTE

≠

CURRENT

DETTAGLIATO

≠

AUTHORITATIVE

RECENTE

≠

APPROVED

VISIBILE IN UI

≠

SOURCE OF TRUTH

Questa è una protezione fondamentale in un'organizzazione agentica, dove grandi quantità di testo possono essere generate rapidamente.

## 12.22 Il ruolo del Canon Register

Una precedence funziona bene soltanto se il sistema sa dove vive l'authority.

Per questo WCM mantiene un Canon Register.

Il registro non duplica tutte le regole.

Indica quali nodi sono attualmente autorevoli per le diverse aree.

In pratica risponde a una domanda fondamentale:

**"Se devo verificare questa parte del metodo, qual è la fonte che devo considerare canonica oggi?"**

È una mappa dell'authority.

Non è una nuova authority separata dalle fonti che elenca.

## 12.23 Procedura mentale minima

Quando WCM incontra più fonti candidate, il controllo può essere espresso così:

```
1. CHE TIPO DI INFORMAZIONE CERCO?  
↓  
2. QUALE FONTE HA AUTHORITY SU QUEL TIPO DI INFORMAZIONE?  
↓  
3. QUAL È IL SUO STATUS CORRENTE?  
↓  
4. ESISTE UNA PRECEDENCE PIÙ SPECIFICA DEL TASK?  
↓  
5. LE FONTI CONFLIGGONO DAVVERO?  
↓  
NO → usa la fonte appropriata  
YES → fail closed / escalation
```

Non serve trasformare ogni consultazione in una cerimonia.

La disciplina deve diventare naturale.

---

## 12.24 Anti-pattern

### Anti-pattern 1 — "È il file più recente"

Errore: confondere recency e authority.

### Anti-pattern 2 — "È quello che ricordo"

Errore: confondere Working Memory e source of truth.

### Anti-pattern 3 — "Lo vedo nel pannello"

Errore: confondere projection e authority.

### Anti-pattern 4 — "Il test dimostra che dobbiamo cambiare"

Errore: confondere evidence e promotion.

### Anti-pattern 5 — "Sono due fonti autorevoli, faccio una media"

Errore: silent conflict resolution.

### Anti-pattern 6 — "Il runtime dice così, quindi può autorizzarlo"

Errore: estendere la precedence del runtime oltre gli execution facts.

---

## 12.25 Una formula compatta

Possiamo riassumere Source Precedence così:

```
NON CHIEDERE SOLO:  
"Quale fonte parla di questo?"  
  
CHIEDI:
```

"Quale fonte ha authority su questa specifica informazione?"

E poi:

```
AUTHORITY
+
STATUS
+
SCOPE
+
TIPO DI FATTO
+
TASK CORRENTE
=
SOURCE PRECEDENCE
```

## 12.26 Cosa abbiamo ottenuto

Con i Capitoli 9-12 abbiamo costruito una catena completa.

```
TROPPIA CONOSCENZA
↓
KNOWLEDGE NAVIGATION LAYER
↓
INDEX-FIRST
↓
PROGRESSIVE RETRIEVAL
↓
SOURCE PRECEDENCE
↓
CONTESTO MINIMO MA AUTOREVOLE
```

Ora WCM non sa soltanto **dove cercare**.

Sa anche **quale fonte deve pesare di più per la domanda corrente**.

Questo prepara il passaggio successivo.

Nel Capitolo 13 non partiremo più da una fonte.

Partiremo da una richiesta.

E vedremo come il WCM la trasforma in una route verso goal, scope, authority, processi, protocolli, guard e capability applicabili.

## Source Map

### Fonti canoniche principali

- [WCM\\_AGENT\\_START.md](#) — bootstrap generale, gerarchia authority/canon/runtime e regole di execution precedence;
- [wcm/GOVERNANCE.md](#) — WCM RUN / WCM CHANGE, authority, gate e natura delle projection;

- [wcm/kb/index.md](#) — Method KB entry point e source map corrente;
- [wcm/kb/canon/CANON\\_REGISTER.md](#) — mappa delle fonti autorevoli correnti;
- [wcm/kb/concepts/CONCEPT-007\\_AGENT\\_READY\\_KNOWLEDGE\\_ARCHITECTURE.md](#) — Source Precedence nel Knowledge Navigation Layer;
- [wcm/process-book/protocols/PROT-005\\_INDEX\\_FIRST\\_PROGRESSIVE\\_RETRIEVAL.md](#) — precedence generale, status matters, no silent conflict resolution, Memory is not authority;
- [wcm/kb/decisions/DEC-013\\_DETERMINISTIC\\_OPERATIONAL\\_STATE\\_PIPELINE.md](#) — precedence specifica per execution facts e projection chain;
- [wcm/documentation/DOCUMENTATION\\_INDEX.md](#) — manuali come living projections e reader come distribution surface.

## Relazioni

```
CH12
├─ DERIVED_FROM → CONCEPT-007
├─ GOVERNED_BY → PROT-005
├─ CONSTRAINED_BY → GOVERNANCE
├─ CONSTRAINED_BY → CANON_REGISTER
├─ SPECIALIZED_BY → DEC-013 per execution facts
└─ CONTINUES → CH09 / CH10 / CH11
```

## Maturity note

La Source Precedence è parte della baseline Agent-Ready implementata e della disciplina INDEX-FIRST corrente. La validazione sul campo del modello complessivo continua; il capitolo non implica validazione universale di efficienza, scalabilità o completezza semantica.